

# OpenONDA

## User Manual

Finite-volume, vortex-particle and vortex-lattice methods for incompressible flow

Operator for Numerical Design and Aerodynamics

October 8, 2026

# Contents


**1 Introduction** **3**
1.1 Choose a model . . . 3

**2 Installation** **4**
2.1 Into an existing Python environment . . . 4
2.2 First simulation . . . 4
2.3 GPU acceleration . . . 4
2.4 Check the installation . . . 5

**3 Learning with the tutorials** **6**
3.1 Anatomy of a case . . . 6
3.2 FVM: viscous flow on a mesh . . . 6
3.3 VPM and VLM: vortex particles and lifting surfaces . . . 7
3.4 Coupled FVM–VPM . . . 7
3.5 Variants . . . 7

**4 Finite-volume flow** **8**
4.1 Physical model and units . . . 8
4.2 Two ways to set up a case . . . 8
4.3 Mesh setup . . . 9
4.4 Boundary conditions . . . 10
4.5 Time and discretisation . . . 10
4.6 Turbulence and LES . . . 11
4.7 Observations . . . 11
4.8 Parallel runs . . . 12

**5 Vortex particles and lifting surfaces** **13**
5.1 Physical model and units . . . 13
5.2 Particle distributions . . . 14
5.3 Induction and stretching . . . 14
5.4 Diffusion and LES . . . 14
5.5 Stabilization . . . 15
5.6 Planar flow . . . 16
5.7 VLM surfaces and wakes . . . 17
5.8 Run and inspect . . . 17

**6 FVM–VPM coupling** **19**
6.1 How an exchange works . . . 19
6.2 Geometry and mesh . . . 19
6.3 Boundary conditions . . . 20
6.4 Vorticity transfer . . . 20
6.5 Configuration . . . 20

**7 Continue or extend a simulation** **22**
7.1 What a backup contains . . . 22
7.2 Keep the case compatible . . . 22

**8 Saved results** **23**

**9 Inspect and compare the flow** **24**
9.1 Reading the fields . . . . . 24

CHAPTER 1

# Introduction

**OpenONDA** (**Operator for Numerical Design and Aerodynamics**) simulates incompressible viscous flows, vortex wakes, wings, and rotors with finite-volume (FVM), vortex-particle (VPM), and vortex-lattice (VLM) methods.

## 1.1 Choose a model

- **FVM:** resolve velocity and pressure on a mesh, including viscous flow around walls.
- **VPM/VLM:** evolve vorticity on particles; use lifting surfaces to generate wing and rotor wakes.
- **FVM–VPM:** resolve the flow near a body with FVM and carry its outer wake with particles.

This manual collects the user guides. Each chapter links to the tutorial cases that demonstrate it; the tutorials themselves live in the `tutorials/` directory of the repository.

CHAPTER 2

# Installation

On Linux x86-64 or macOS (Intel or Apple Silicon), use Bash or Zsh:

```
git clone --depth 1 --branch development https://github.com/EngFlavioMartins/OpenONDA.git
cd OpenONDA
source install.sh
```

The installer creates and activates the **OpenONDA** Conda environment, installs Python 3.11 and the solver, meshing, MPI, and plotting dependencies, and checks the installation. If Conda is absent, it installs Miniforge. Administrator access is not required.

In a new terminal:

```
conda activate OpenONDA
```

Keep the checkout in place: installation is editable, so edits to the code take effect immediately. Re-run `source install.sh` to update the environment, and add `--dev` to also install the test, lint and packaging tools. The installer adds Conda's start-up lines to your shell only when it had to install Conda itself.

After installation, `import openonda` works from any directory; cases need not live inside the checkout.

## 2.1 Into an existing Python environment

With your own CPython 3.11 environment, install from the checkout with pip:

```
pip install -e .
```

Optional extras add features: `[parallel]` for MPI runs (mpi4py and petsc4py; Conda supplies a consistent MPI more reliably), `[meshing]` for Gmsh import, `[gaussian-mesh-cuda12]` for the CUDA image sums below, `[test]` or `[dev]` for the test suite and developer tools, for example `pip install -e ".[parallel,meshing]"`. Tutorial plots also need a LaTeX installation, which `source install.sh` provides.

## 2.2 First simulation

```
cd tutorials/fvm/taylor_green
./allrun.sh
./allplot.sh
```

This case follows viscous decay of a periodic vortex. Choose other flows from the tutorial index, or configure a case with the FVM, VPM/VLM, or coupling guide.

## 2.3 GPU acceleration

The standard installation runs on the CPU. Particle cases run on a GPU when `compute_device` selects a device with suitable drivers (`CUDA`, `VULKAN` or `METAL`; `AUTO` picks one). Slip-slab cases,

including planar flow, evaluate their mirror images fastest with the optional CUDA extension, which supplies CuPy and matching CUDA 12 libraries for an NVIDIA GPU:

```
python install.py --gaussian-mesh-cuda12
```

Run it in the activated environment. Without it, these image sums fall back to a slower CPU evaluation.

## 2.4 Check the installation

```
openonda info
```

prints the installed version and location; `openonda api <name>` shows the documentation of any public class or function, for example `openonda api openonda.vpm.ViscousConfig`.

CHAPTER 3

# Learning with the tutorials

The tutorials are the main way to learn OpenONDA. As in OpenFOAM, each case is a small, self-contained directory that sets up one physical problem. Run it as it is, compare with the reference it ships, then change it. Work through them roughly in the order below; each step adds a few new solver features. The case directories are under `tutorials/fvm`, `tutorials/vpm` and `tutorials/coupled_fvm_vpm`.

## 3.1 Anatomy of a case

```
case/
├── setup.py        physical constants, mesh or particles, solver settings
├── allrun.sh       clean, then run the case (and its variants)
├── allcontinue.sh  resume from the latest backup
├── allplot.sh      write the figures
├── allclean.sh     delete solution/, samples/ and figures/
├── assets/         geometry, initial conditions, plot scripts, ParaView state
└── README.md       the physics, the parameters and what to look for
```

`setup.py` begins with the physical inputs in SI units: lengths in m, velocities in m/s, kinematic viscosity in $m^2/s$, times in s. Below them, one function builds the solver configuration, and the last lines run it. To make your own variant, copy the directory and edit the constants:

```
cp -r tutorials/vpm/02_vortex_ring ~/ring && cd ~/ring
./allrun.sh && ./allplot.sh
```

When checking convergence, refine the spacing and the time step together. Results go to `solution/` and `samples/`, figures to `figures/`; see saved results, continuation and visualization.

## 3.2 FVM: viscous flow on a mesh

| Case | Physics | You learn |
|---|---|---|
| Taylor–Green vortex | Decaying periodic vortices; exact solution | Cyclic boundaries, time and convection schemes, numerical dissipation |
| Flat-plate boundary layer | Laminar flat plate; Blasius theory | Stretched wall grids, inlet/outlet/wall, skin friction |
| Backward-facing step | Backward-facing step | Separation, reattachment, adaptive time step |
| Square cylinder | Square-cylinder shedding | Force coefficients, Strouhal number |
| STL body | Flow around an STL body | Octree meshing, refinement regions, slip walls |
| NACA 0012 airfoil | NACA 0012 airfoil | Far-field boundaries, surface pressure |

## 3.3 VPM and VLM: vortex particles and lifting surfaces

| Case | Physics | You learn |
|---|---|---|
| Lamb–Oseen vortices | Viscous vortex, dipole and merger | Building particle fields; CS, RWM, DVH and GBD diffusion |
| Vortex ring | Perturbed vortex ring | Stretching schemes, LES, state limits |
| Leapfrogging rings | Leapfrogging rings | Stabilization: eddy viscosity, strength alignment, splitting, regularization |
| Flat plate | Inclined flat plate | Lifting surfaces, startup loads, wake shedding, moving frames |
| Heaving delta wings | Two heaving wings | Prescribed motion, wake–wing interaction |
| Wind turbine | Wind turbine | Rotating blades, power and thrust, LES wake, filament splitting |
| Quadcopter | Four rotors in climb | Interacting rotor wakes, the Winckelmans kernel |

## 3.4 Coupled FVM–VPM

| Case | Physics | You learn |
|---|---|---|
| Cylinder wake | Laminar shedding at $Re = 150$ | Coupling setup, planar flow with one particle layer, outflow fringes, a sensitivity study |
| Cube wake | Separated flow at $Re = 1000$ | Three-dimensional coupling, LES in both solvers |

Each coupled case has a `reference_flow/` subdirectory that solves the same flow with the FVM alone on a large domain; the cube reference also shows a grid-convergence study and MPI parallel runs.

## 3.5 Variants

Several cases run variants from one `setup.py`, listed one per line in `allrun.sh`: diffusion schemes (`python setup.py vortex GBD`), stretching schemes (`--variant dns_transposed`), stabilization mechanisms (`python setup.py baseline`), plate angles (`--angle -5`), grid levels (`--name grid_h0045 -h 0.045`), or coupling parameters (`--box-margin 1.4`). Run a single variant by copying its line. Each variant writes and resumes its own results.

CHAPTER 4

# Finite-volume flow

The finite-volume method (FVM) solves constant-density incompressible flow on a static mesh. Use it where walls, pressure and thin shear layers must be resolved. The tutorials teach it step by step:

| To learn | Start with |
|---|---|
| Periodic flow, time and convection schemes | Taylor–Green vortex |
| Wall-resolved laminar flow, skin friction | Flat-plate boundary layer |
| Inlet/outlet flow, separation and reattachment | Backward-facing step |
| Bluff-body shedding, force coefficients | Square cylinder |
| Meshing around an STL body | STL body |
| Far-field boundaries, surface pressure | NACA 0012 airfoil |
| LES, MPI parallel runs, grid studies | Cube reference flow |

## 4.1 Physical model and units

The velocity $\mathbf{U}$ and kinematic pressure $q = p/\rho$ satisfy

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U} \otimes \mathbf{U}) = -\nabla q + \nabla \cdot (\nu_{\text{eff}} \nabla \mathbf{U}), \qquad \nabla \cdot \mathbf{U} = 0,$$

with $\nu_{\text{eff}} = \nu + \nu_t$. Set the density $\rho$ and molecular viscosity $\nu$ in `TransportConfig`; for a target Reynolds number use $\nu = U_{\text{ref}} L_{\text{ref}} / Re$. All inputs are SI:

| Quantity | Unit | Meaning |
|---|---|---|
| `density` | kg/m³ | $\rho$; converts kinematic pressure and forces to Pa and N |
| `kinematic_viscosity` | m²/s | Molecular viscosity $\nu$ |
| `velocity` | m/s | Cell-centred $\mathbf{U}$ |
| `kinematic_pressure` | m²/s² | $q$; multiply by $\rho$ for pressure in Pa |
| `eddy_viscosity` | m²/s | Subgrid viscosity $\nu_t$ |
| `vorticity` | 1/s | $\omega = \nabla \times \mathbf{U}$ |
| `time_step_size`, `end_time` | s | Time resolution and run duration |
| Mesh coordinates, cell sizes | m | Geometry |

## 4.2 Two ways to set up a case

`FVMCase` groups a mesh, `Numerics`, boundaries, initial fields and a `RunPlan`, and runs through `FVMSolver`:

```
import numpy as np

from openonda import fvm

x = np.linspace(0.0, 2.0, 17)  # m, 16 cells along x and y
mesh = fvm.mesher.box_mesh_3d(x, x, np.array([0.0, 0.125]), empty_spanwise=True)

case = fvm.FVMCase(
    name="uniform-flow",
    directory="first-fvm-case",
    mesh=mesh,
    numerics=fvm.Numerics(
        transport=fvm.TransportConfig(density=1.0, kinematic_viscosity=0.01),
        coupling=fvm.PimpleControl(algorithm="PISO", n_correctors=2),
    ),
    boundaries=(
        fvm.BoundaryConfig.inlet("inlet", [1.0, 0.0, 0.0]),
        fvm.BoundaryConfig.outlet("outlet"),
        fvm.BoundaryConfig.slip("ymin"),
        fvm.BoundaryConfig.slip("ymax"),
        fvm.BoundaryConfig.empty("zmin"),
        fvm.BoundaryConfig.empty("zmax"),
    ),
    initial_conditions=fvm.InitialFields(velocity=(1.0, 0.0, 0.0)),
    run=fvm.RunPlan(end_time=0.1, time_step_size=1.0e-3),
)
with fvm.FVMSolver(case) as solver:
    solver.run()
```

The tutorials use the equivalent flat form: an `FVMSetup` holding `time`, `schemes`, `linear`, `pimple`, `transport`, `boundaries`, `samplers` and `turbulence`, passed with the mesh to `fvm.create_fvm_solver(setup, case_dir=..., mesh=...)`. Both accept the same configuration objects.

## 4.3 Mesh setup

`openonda.fvm.mesher` builds meshes in metres:

| Tool | Use |
|---|---|
| `box_mesh_3d(xs, ys, zs, hole_box=...)` | Rectilinear mesh from node coordinates, optionally with a box-shaped wall hole. `empty_spanwise=True` with one spanwise cell gives a 2-D model. |
| `stretched(...)`, `wall_refined_axis(...)` | Node sequences that grow geometrically away from a wall, for `box_mesh_3d`. |
| `coupling_box_mesh(...)` | Uniform box whose six sides form one coupling patch (FVM–VPM). |
| `CartesianMesher` | Octree mesh of a `BoxDomain` around closed `STLSurface` bodies, projected onto the surfaces. |
| `ExtrudedCartesianMesher` | Planar section of a `CartesianMesher` mesh, extruded through given span `levels`: body-fitted 2-D meshes. |

| Tool | Use |
|---|---|
| `GmshImporter` | Converts a Gmsh mesh. |

`CartesianMesher` refines with `BoxRefinement`, `SphereRefinement`, `ConeRefinement` and `LineRefinement` regions, `PatchRefinement` on a named surface, `FeatureRefinement` along sharp edges, and `BoundaryLayers` for wall-normal prism layers. Cell sizes are $H/2^{\ell}$ with $H$ the background `max_cell_size`: a patch target selects the first size at or below the request, a region target a size strictly below it. `CartesianMesher.effective_cell_size(target)` previews the result. The STL body shows a complete mesher; the cylinder shows an extruded body-fitted 2-D mesh.

A generated mesh is saved next to the case as `assets/<case_name>_mesh.npz` (with a `.vtu` copy for ParaView) and reused while its inputs are unchanged. The solver checks cell volumes and closure on construction.

Refine wall-normal spacing where viscous shear matters; the flat plate uses a stretched wall-normal grid.

## 4.4 Boundary conditions

Every mesh patch needs a `BoundaryConfig` with the same name.

| Constructor | Physical condition | Tutorial |
|---|---|---|
| `inlet(name, [U, V, W])` | Prescribed velocity, zero normal pressure gradient | Backward-facing step |
| `outlet(name, kinematic_pressure=0)` | Pressure datum, zero-gradient outflow velocity | Square cylinder |
| `wall(name)` | Stationary no-slip wall | Flat plate |
| `slip(name)` | Impermeable, shear-free boundary | STL body |
| `freestream(name, [U, V, W])` | Far field: inflow takes the freestream velocity, outflow the pressure datum | Airfoil |
| `cyclic(name, neighbour)` | Periodic pair; declare both directions | Taylor–Green |
| `empty(name)` | Span faces of a one-cell 2-D mesh | Backward-facing step |

A time-dependent inflow uses `VelocityRamp(initial=..., final=..., start_time=..., end_time=...)`, attached to patches through `FVMSetup(velocity_boundaries=(VelocityBoundary(...),))`. With `normal_only=True` the ramp prescribes only the normal velocity, so a slip patch can carry a transient transverse inflow; the cylinder reference uses one to trigger shedding.

## 4.5 Time and discretisation

| Setting | Options |
|---|---|
| `PimpleControl(algorithm=...)` | `PISO`: transient, `n_correctors` pressure corrections. `PIMPLE`: adds `n_outer_correctors` nonlinear corrections, with `velocity_relaxation`/`pressure_relaxation`. `SIMPLE`: steady state through `solver.solve_steady()`. Each PISO correction updates the momentum prediction with the previous correction's velocity change (Issa 1986). |
| `DiscretizationConfig(time_scheme=...)` | `euler_implicit` (first order) or `backward` (second-order BDF). |
| `convection_scheme` | `upwind` (robust, dissipative), `central` (accurate when resolved, may oscillate), bounded `limitedLinear` (default), `linearUpwind`, `LUST`, `vanLeer`, `MUSCL`, `minmod`, `superbee`. |
| `gradient_scheme` | `lsq` (default) or `gauss`. |
| `TimeConfig(adjustment=MaximumCourantTimeStep(maximum_time_step_size))` | Adapts the step to a Courant limit. |

`LinearSolverConfig` sets the momentum and pressure solver tolerances; `ComputeConfig(operator_backend="numba")` speeds up the discrete operators. Repeat a case with finer cells and a smaller step before treating drag, shedding frequency or wall shear as converged; `fvm.update_grid_study` and `fvm.analyse_grid_study` estimate the discretisation error from a grid family, as in the cube reference flow.

## 4.6 Turbulence and LES

The default `TurbulenceConfig.none()` sets $\nu_t = 0$, which the laminar tutorials use. For large-eddy simulation, set `turbulence` to one of:

| Model | Closure and default coefficients |
|---|---|
| `smagorinsky()` | $\nu_t = (C_s\Delta)^2\lvert S\rvert$, $C_s = 0.17$ |
| `equilibrium_smagorinsky()` | Subgrid energy in local equilibrium, $C_k = 0.094$, $C_e = 1.048$ |
| `dynamic_smagorinsky()` | Germano–Lilly coefficient from a test filter, globally averaged |
| `wale()` | WALE, $C_w = 0.325$; vanishes near walls without damping |
| `sigma()` | Singular values of the velocity gradient, $C_\sigma = 1.35$ |

Here $\lvert S\rvert = \sqrt{2S_{ij}S_{ij}}$ and $\Delta = V^{1/3}$, or $\sqrt{V/b}$ on a one-cell 2-D mesh of thickness $b$. Refining the mesh changes both the resolved scales and the modelled viscosity. WALE or sigma suit wall-bounded flows without a homogeneous direction. The cube cases use equilibrium Smagorinsky.

## 4.7 Observations

Samplers write CSV histories to `samples/` on a `RunSchedule(every_time=...)` or `every_n_steps`:

| Sampler | Output |
|---|---|
| `ForceSampler(patch_names, reference_velocity, reference_area, reference_length)` | Pressure and viscous forces, moments and coefficients $C_D = F_x/(\frac{1}{2}\rho U^2 A)$ |
| `LineSampler(start, end, n_points=...)` | Velocity, vorticity and pressure along a line |
| `SurfaceSampler(point, normal, bounds, spacing)` | The same fields on a planar grid |
| `YPlusSampler(patch_names)` | Wall $y^+$ statistics |

`fvm.compute_kinetic_energy`, `compute_enstrophy` and `compute_continuity_error` evaluate global diagnostics from saved fields. Volume fields are written to `solution/fvm.pvd` for ParaView; see results.

## 4.8 Parallel runs

Set `FVMSetup(cores=N)`. The solver launches its own MPI workers, switches to PETSc linear algebra and partitions the mesh (cyclic cases keep a replicated mesh). Partitioning pays off on meshes of a few hundred thousand cells or more; small meshes run faster serially. A partitioned backup restarts only with the same core count.

CHAPTER 5

# Vortex particles and lifting surfaces

The vortex-particle method (VPM) follows vorticity instead of velocity. Each particle carries a small, smoothed blob of vorticity; the particles move with the flow they induce, their strengths stretch and tilt, and viscosity diffuses them. Lifting surfaces, represented by a vortex-lattice model (VLM), shed their circulation into the particle wake. There is no volume mesh, so a wake can travel far from the body at no extra cost.

| To learn | Start with |
|---|---|
| Building particle fields; the four diffusion schemes | Lamb–Oseen vortices |
| Vortex stretching formulations; LES | Vortex ring |
| Stabilization and particle regularization | Leapfrogging rings |
| Lifting surfaces, startup loads and wakes | Flat plate |
| Prescribed wing motion | Heaving delta wings |
| Rotating blades, wake refinement | Wind turbine, quadcopter |
| Planar flow with one particle layer | Coupled cylinder |

## 5.1 Physical model and units

For constant viscosity, the vorticity $\omega = \nabla \times \mathbf{u}$ obeys

$$\frac{D\omega}{Dt} = (\omega \cdot \nabla)\mathbf{u} + \nu \nabla^2 \omega,$$

and the velocity follows from the vorticity through the Biot–Savart law. A particle at $\mathbf{x}_p$ carries the vector strength $\Gamma_p = \omega_p V_p$: the vorticity integrated over its volume $V_p$. A radial kernel of core radius $\sigma_p$ spreads it in space, which keeps the induced velocity finite.

| Quantity | Symbol | Unit |
|---|---|---|
| `position` | $\mathbf{x}$ | m |
| `velocity`, `freestream_velocity` | $\mathbf{u}$ | m/s |
| `vortex_strength` | $\Gamma_p = \omega_p V_p$ | m³/s |
| `vorticity` | $\omega$ | 1/s |
| `particle_volume` | $V_p$ | m³ |
| `core_radius` | $\sigma$ | m |
| `spacing` | $h$ | m |
| `kinematic_viscosity`, `eddy_viscosity` | $\nu$, $\nu_t$ | m²/s |
| `time_step_size` | $\Delta t$ | s |

A filament or ring circulation (m²/s) and its physical core radius belong to the flow you model; particle strength and particle core radius belong to its discretisation.

## 5.2 Particle distributions

`VPMCase.initial_conditions` holds objects whose `build()` returns a `vpm.VortexParticleSet`: positions, volumes, core radii, strengths $\omega V_p$, viscosity, the nominal spacing and optional `group_id` labels that tag each vortex in diagnostics. The tutorials build theirs in `assets/initial_conditions.py`: Gaussian filaments on a triangular lattice for the Lamb–Oseen vortices, and Gaussian rings on a toroidal lattice with Widnall disturbances for the single and leapfrogging rings. Copy one as a template; the run example builds a vortex column in a few lines.

Choose $h$ fine enough to resolve the smallest vortex core, and $\sigma/h$ above one so that neighbouring blobs overlap; `core_radius_ratio` in the viscous settings sets it (default 2.5). Large cores smooth small structures; small cores leave holes between particles. Because the blobs themselves smooth the field, a Gaussian vortex of radius $a$ is sampled with the remaining width $\sqrt{a^2 - \sigma^2}$, which needs $a > \sigma$. Refine $h$ and $\sigma$ together and compare physical observables.

## 5.3 Induction and stretching

| Setting | Options |
|---|---|
| `particle_kernel` | `GAUSSIAN` (default), `HIGH_ORDER_GAUSSIAN`, `SUPER_GAUSSIAN`, `WINCKELMANS` (algebraic). |
| `induction` | `DirectInduction()`: exact sum over all pairs, for small or reference cases. `TreecodeInduction(theta=..., multipole_order=...)`: hierarchical far field for Gaussian and Winckelmans kernels; a smaller `theta` is more accurate. `FMMInduction()`: fast multipole method, every kernel. `SlipSlabInduction(base, z_min, z_max)`: free-slip span planes, see planar flow. |
| `stretching_scheme` | `TRANSPOSED` (default) uses $J^T\Gamma$ and conserves total strength; `DIRECT` uses $J\Gamma$; `MIXED` uses the symmetric part $S\Gamma$. The vortex ring compares them. |
| `integrator` | `RK2()`, `SSPRK3()` or `RK4()`, advancing positions and strengths together. |
| `compute_device`, `precision` | `AUTO`, `CPU`, `CUDA`, `VULKAN` or `METAL`; `f32` or `f64`. |

Check the fast methods against `DirectInduction` on a small case before trusting forces or instability growth. `freestream_velocity` adds a uniform background flow.

## 5.4 Diffusion and LES

| `ViscousConfig` | Physical model | Tutorial |
|---|---|---|
| `cs()` | Core spreading: $d\sigma^2/dt = 4\nu_{\text{eff}}$, exact heat spreading of a Gaussian. Cores grow, so long runs need regularization. | Ring, flat plate |

| `ViscousConfig` | Physical model | Tutorial |
|---|---|---|
| `rwm()` | Random walk: Brownian steps of variance $2\nu\Delta t$. Uniform viscosity; average an ensemble. | Lamb–Oseen |
| `dvh()` | Diffused vortex hydrodynamics: heat-kernel spreading onto a grid, then regeneration. Uniform viscosity. | Lamb–Oseen |
| `gbd()` | Grid-based diffusion: strengths diffuse on a grid with an explicit Laplacian, then regenerate as particles. Supports variable viscosity and LES. | Lamb–Oseen, coupled cases |
| `inviscid()` | No diffusion. | — |

The grid schemes (`dvh`, `gbd`) need a `padding` around the particles, in grid cells, and a vorticity `threshold` below which regenerated particles are dropped (`threshold_mode="absolute"` in $m^3/s$, or a `"budget"` fraction of the total).

`TurbulenceConfig.dns()` adds no model. `les_smagorinsky()` and `equilibrium_smagorinsky()` add an eddy viscosity

$$\nu_t = (C_s\Delta)^2\sqrt{2S:S}, \qquad \Delta = V_p^{1/3}, \qquad \nu_{\text{eff}} = \nu + \nu_t,$$

with $C_s = 0.20$ by default; `filter_width` fixes $\Delta$ in metres. LES works with core spreading and grid-based diffusion. See the ring for a DNS/LES comparison and the wind turbine for an LES wake.

## 5.5 Stabilization

Particles drift apart and stretch, so a long run gradually loses resolution. `StabilizationConfig` gathers the remedies. Each changes the numerical evolution, so compare with a baseline as the leapfrogging rings do.

| Mechanism | Effect |
|---|---|
| `bounded_domain(bounds)` | Removes particles that leave a keep box; `outflow_fringe_depth` fades them out first (below). |
| `selective_eddy_viscosity(coefficient)` | Damps small-scale noise only. |
| `pedrizzetti_relaxation(factor, interval_steps)` | Aligns strengths with the local vorticity. |
| `conservative_filter(...)` | Regenerates the particles on a regular lattice at intervals, conserving total strength and impulse within energy and enstrophy loss limits. |
| `filament_refinement` | Splits over-stretched particles along their vortex lines: `FilamentRefinementConfig.adaptive(interval_steps=...)`. |
| `divergence_relaxation` | Reduces the divergence of the particle vorticity field: `DivergenceRelaxationConfig(...)`. |

Deleting a shed vortex abruptly at the domain face changes the circulation left behind, which induces a spurious alternating velocity at the body. Outflow fringes instead damp strengths smoothly over a depth $L$ inside a face,

$$\frac{d\Gamma}{dt} = -\lambda(s)\Gamma, \qquad \lambda = \frac{2A\|\mathbf{U}_\infty\|}{L} s^3(10 - 15s + 6s^2),$$

with $s \in [0, 1]$ the normalized depth and $A$ the number of e-folds (`outflow_fringe_attenuation`, 8 by default). Make a fringe at least two wake wavelengths deep, as in the coupled cylinder.

`ParticleStateLimits` checks every accepted step: finite values, a Lagrangian Courant limit, maximum strength, divergence, strength–vorticity misalignment and growth rates. `RunPlan(state_limit_action="STOP")` ends a run cleanly at a crossed limit instead of raising an error.

## 5.6 Planar flow

`SlipSlabInduction(base, z_min=..., z_max=...)` bounds the span with two free-slip planes. Each particle then has an infinite row of mirror images: a reflection keeps the spanwise strength and reverses the in-plane components. One particle layer between planes one spacing apart, with all its images, induces the velocity of planar vortices; a Gaussian layer gives planar Gaussian vortices. There is no separate two-dimensional solver: the three-dimensional operators act on the image-complete field, so every kernel, integrator, diffusion scheme and stabilization mechanism carries over.

```
h = 0.04  # particle spacing, m
span = (-0.5 * h, 0.5 * h)  # slip planes, m
plane = (-5.0, 30.0, -5.0, 5.0)  # x and y bounds, m

numerics = vpm.Numerics(
    induction=vpm.SlipSlabInduction(vpm.FMMInduction(), z_min=span[0], z_max=span[1]),
    viscous=vpm.ViscousConfig.gbd(particle_spacing=h, kinematic_viscosity=nu),
    domain_bounds=(*plane, *span),
    stabilization=vpm.StabilizationConfig.bounded_domain((*plane, *span)),
)
```

Place the particles on $z = 0$ with volume $h^3$ and spanwise strength $\Gamma_z = \omega_z h^3$. Then:

- repeat the planes in `domain_bounds` and in any keep box, and keep z-face fringe depths at zero;
- make the slab width a multiple of half the spacing of any grid in use (diffusion, regularization), so the planes fall on nodes or half nodes;
- with `dvh()`, set `padding` to at least `dvh_support_radius_ratio`; LES uses `gbd()`;
- for the algebraic Winckelmans kernel, use $\sigma \geq 2h$: a discrete layer of algebraic cores ripples along the span.

`Numerics` rejects inconsistent settings with a message naming both values. The distant images are summed in closed form with a rigorous error bound checked against `tail_tolerance`, so even one layer needs only a few image shells. Thicker slabs model a spanwise-periodic segment between free-slip walls.

## 5.7 VLM surfaces and wakes

A lifting surface is a lattice of vortex rings whose circulation cancels the normal velocity on the surface; changes of bound circulation are shed into the particle wake.

- **Geometry:** a JSON surface of quadrilateral segments with vertices `a,b` on the leading edge and `d,c` on the trailing edge, and `n_chordwise_panels`, `n_spanwise_panels`. The flat-plate setup writes one; `VLMMeshSetup.geometric(ratio, region)` clusters panels toward an edge.
- **Placement:** `VLMSurfaceSetup(surface, translation=..., rotation_degrees=..., rotation_centre=..., kinematics=...)`, collected in `Numerics(vlm=VLMSetup(surfaces=(...), density=..., kinematic_viscosity=...))`.
- **Motion:** `StaticVLM`, `TranslatingVLM`, `SmoothRampVLM`, `RotatingVLM`, `PitchingVLM`, or `ManeuverVLM` driven by your own velocity functions.
- **Loads:** `ForceConfig.kutta_joukowski(unsteady=True)` adds the unsteady pressure load to the bound-vortex forces; `VLMSampler` writes surface geometry, circulation and loads.
- **Wake:** `wake_core_overlap` (default 2.5) sets the core of shed particles relative to the panel and row size; refine $\Delta t$ with the panels.

VLM assumes attached, inviscid flow: it does not model boundary layers, skin friction, stall or leading-edge separation. Progress from the flat plate to moving wings, the wind turbine and the quadcopter.

## 5.8 Run and inspect

This example advances a Gaussian vortex column ($\Gamma = 1$ m²/s, core radius 0.35 m, $\nu = 0.01$ m²/s) by two steps:

```
import numpy as np

from openonda import vpm

h, a, nu = 0.2, 0.35, 0.01  # particle spacing [m], core radius [m], viscosity [m²/s]


class GaussianColumn:
    """Vortex column of unit circulation along z, sampled on a cubic lattice."""

    def build(self):
        x = np.arange(-1.0, 1.0 + h / 2, h)
        position = np.stack(np.meshgrid(x, x, x, indexing="ij"), axis=-1).reshape(-1, 3)
        n = len(position)
        radius_squared = position[:, 0] ** 2 + position[:, 1] ** 2
        strength = np.zeros((n, 3))
        strength[:, 2] = np.exp(-radius_squared / a**2) / (np.pi * a**2) * h**3  # omega * V
        return vpm.VortexParticleSet(
            position=position,
            velocity=np.zeros((n, 3)),
            vortex_strength=strength,
            core_radius=np.full(n, 1.2 * h),
            particle_volume=np.full(n, h**3),
            kinematic_viscosity=np.full(n, nu),
            spacing=h,
        )
```

```
case = vpm.VPMCase(
    directory="first-vpm-case",
    numerics=vpm.Numerics(
        time_step_size=0.01,
        compute_device="CPU",
        max_n_particles=5000,
        integrator=vpm.RK2(),
        induction=vpm.DirectInduction(stretching_scheme="TRANSPOSED"),
        viscous=vpm.ViscousConfig.cs(kinematic_viscosity=nu, particle_spacing=h),
    ),
    initial_conditions=(GaussianColumn(),),
    backup=vpm.Backup(interval_steps=10),
    samplers=vpm.Samplers(samples=(
        vpm.FlowIntegralsSampler(schedule=vpm.EverySteps(10)),
    )),
    run=vpm.RunPlan(steps=2),
)
vpm.VPMSolver(case).run(start_from="latest")
```

`RunPlan.steps` is the total number of steps, so a resumed run continues to it; see continuation. Set `max_n_particles` for the largest cloud the run will reach.

Samplers write to `samples/` on an `EverySteps`, `EveryTime` or `FinalOnly` schedule. `FlowIntegralsSampler` records the global invariants per unit density: total strength, linear and angular impulse, kinetic energy and enstrophy. `LineSampler` and `SurfaceSampler` record velocity and vorticity on probes. Particles are written to `solution/vpm.pvd` for ParaView.

Numerical references.

CHAPTER 6

# FVM–VPM coupling

The coupled solver splits the flow where each method is strongest. The finite-volume solver resolves the body, its no-slip boundary layer, the pressure and the near wake. Vortex particles carry the outer wake, which may travel far downstream without a mesh. Inside an overlap region both describe the same flow: the particles supply the outer boundary condition of the FVM box, and the FVM vorticity replaces the particles near the body.

| Case | Physics |
|---|---|
| Cylinder | Laminar planar shedding at $Re = 150$; one FVM cell and one particle layer across the span. Includes a parameter sensitivity study. |
| Cube | Separated three-dimensional flow at $Re = 1000$ with LES in both solvers. |

Each case has an FVM-only `reference_flow` that solves the same problem on a large domain. Read the FVM and VPM guides first.

## 6.1 How an exchange works

Both solvers advance in steps of the exchange interval $\Delta t_{\mathrm{VPM}} = n\,\Delta t_{\mathrm{FVM}}$:

1. The particles advance one step.
2. The particles' velocity and its normal derivative at the FVM box faces become the FVM boundary condition, and the FVM advances $n$ steps.
3. In the transfer region inside the box, the FVM vorticity replaces the particle representation; outside it the particle wake is kept.
4. Steps 2–3 repeat until the boundary trace converges (`interface_iterations`, tolerances `interface_normal_tolerance` in m/s and `interface_gradient_tolerance` in 1/s). Usually one pass suffices.

Match the freestream, viscosity and turbulence model in both solvers, and define $Re = U_\infty L/\nu$ with the same body length.

## 6.2 Geometry and mesh

1. **FVM box.** Mesh a box around the body with a no-slip wall and an outer patch named `numericalBoundary`. Extend it beyond the vortex-formation region, where separated shear layers are still only a few particle spacings thick: particle cores smooth vorticity over their radius, so an interface cutting those layers biases the near wake. In the cylinder case, moving the downstream face from 2.4 to 4.4 diameters brought the lift amplitude from 6.7% to about 1% of the reference.
2. **Transfer region.** `transfer_region_bounds` sets a box in metres inside the FVM box where the FVM owns the vorticity. Keep the wall-generated vorticity resolved inside it.
3. **Resolution.** Use equal FVM cell size and particle spacing, as both tutorials do, and refine them together.

4. **Particle domain.** Give the particles room for the wake you want to observe, and fade the wake out through an outflow fringe at least two wake wavelengths deep. Removing particles abruptly at a face changes the circulation left in the domain once per shedding half-cycle, which shakes the flow at the body.
5. **Reference.** Compare against a flow with the same boundary conditions. Free-space particles describe an unbounded flow; walls or an inlet a few diameters from the body add blockage (at the cylinder, walls at $\pm 10D$ raise the drag by about 3%).

### 6.2.1 Free-slip span

For a spanwise-periodic or planar flow, bound the particles with `vpm.SlipSlabInduction` and give the FVM box, the particle domain and the transfer region the same span. A single particle layer between planes one spacing apart, with periodic FVM span faces, models planar flow; see planar flow. The coupler stops before the first step if the spans disagree.

## 6.3 Boundary conditions

The particles impose the normal velocity and the tangential part of its normal derivative on the FVM box, following Billuart, Duponcheel, Winckelmans and Chatelain, *J. Comput. Phys.* 473 (2023) 111726:

$$\mathbf{U}_{\mathrm{FVM}} \cdot \mathbf{n} = \mathbf{U}_{\mathrm{VPM}} \cdot \mathbf{n}, \qquad (\mathbf{I} - \mathbf{nn})\, \partial_n \mathbf{U}_{\mathrm{FVM}} = (\mathbf{I} - \mathbf{nn})\, \partial_n \mathbf{U}_{\mathrm{VPM}}.$$

The tangential velocity itself stays free, so the FVM can carry its own boundary-layer and wake structure to the faces. The pressure takes the flux-compatible `fixedFluxPressure` condition. `coupling_patch` names the patch (default `numericalBoundary`); keep walls and slip faces on separate patches.

## 6.4 Vorticity transfer

The particles near the body are renewed from the FVM vorticity at every exchange on a uniform lattice, with grid-based diffusion and M4′ remeshing in the particle solver.

- `eta_blend_width` (m): over this inward width the FVM weight rises from zero to one; the tutorials use $6h$.
- `vpm_only_width` (m): a band just inside the transfer faces that stays purely particle-owned; the tutorials use $2h$.
- `transfer_vorticity_cutoff` (1/s): the smallest vorticity renewed inside the region.
- `transfer_amplification_cap`: limits how strongly a particle strength may be corrected when the particles cannot represent a sharp near-wall target.

Renewal corrects the difference between the FVM vorticity and the field the particles represent, so a field the particles already reproduce is left unchanged, and total strength and impulse are conserved.

## 6.5 Configuration

`FVMSetup`, `VPMCase` and the mesh define the flow; `CouplerSetup` defines the overlap, the convergence controls and the backup interval. `CouplerSetup.freestream` may ramp the background velocity with a `coupler.VelocityRamp`, and `solver.run(initial_velocity=...)` adds an initial disturbance. The cube case uses:

```
from openonda import coupler

cfg = coupler.CouplerSetup(
    freestream_velocity=[1.0, 0.0, 0.0],
    coupling_patch="numericalBoundary",
    transfer_region_bounds=(-1.45, 1.45, -1.45, 1.45, -1.45, 1.45),
    eta_blend_width=6 * 0.045,
    vpm_only_width=2 * 0.045,
    interface_iterations=3,
)
with coupler.create_coupler(FVM_SETUP, VPM_CASE, cfg, mesh=FVM_MESH) as solver:
    solver.run(start_from="latest")
```

The FVM step must divide the exchange step, and output intervals must fall on exchanges. Read forces and profiles together with `solution/coupler_diagnostics.jsonl`, which records the boundary-trace convergence, particle counts and timings per exchange. Before trusting a result, refine the cell and particle spacing, the exchange step and the box size; the cylinder's sensitivity study shows how.

CHAPTER 7

# Continue or extend a simulation

Every tutorial starts its solver from the latest backup (`start_from="latest"`, set in `setup.py` directly or through a `START_FROM` constant): if a numerical backup exists the run resumes from it, otherwise it starts from the initial conditions. From a case directory:

```
./allrun.sh        # delete solution/, samples/ and figures/, then run from t = 0
./allcontinue.sh   # resume from the latest backup
```

A finished run adds nothing when resumed. To extend it, raise the end time in `setup.py` (`END_TIME` for FVM and coupled cases) or the **total** step count (`RunPlan.steps`) for VPM cases, then run `./allcontinue.sh`. Outputs written after the restored time are trimmed first, so the histories stay continuous. Regenerate the figures afterwards with `./allplot.sh`.

In your own script, pass `start_from` to `run()`:

| `start_from` | Behaviour |
|---|---|
| `"latest"` | Resume the most recent backup in `solution/`, or start fresh |
| `"initial"` | Start a new history from the initial conditions, replacing earlier output |
| a path | Resume from that backup |

## 7.1 What a backup contains

| Solver | Backup |
|---|---|
| FVM | `solution/backup`: velocity, pressure and fluxes. An MPI run restarts with the same core count. |
| VPM/VLM | The latest `solution/vpm/vpm_*.h5`: particles and lifting-surface state. |
| Coupled | `solution/backups/`: both solver states and the coupling state; keep the folder together. |

The backup interval sets how much work an interruption costs. ParaView files alone cannot restore a run.

## 7.2 Keep the case compatible

A backup restores the numerical state of one physical problem. Keep the mesh, geometry, boundary conditions, viscosity and numerical model unchanged when resuming; the solver refuses a backup whose configuration differs. To compare a variant, copy the case to a new directory. Backups written by a different version of the solver may need that version to restart.

CHAPTER 8

# Saved results

A case writes its results next to its `setup.py`:

| Location | Contents |
|---|---|
| `solution/fvm.pvd` | FVM velocity, kinematic pressure, vorticity and eddy viscosity at saved times |
| `solution/vpm.pvd` | Particle positions, strengths, vorticity and core radii |
| `solution/vlm.pvd` | Lifting-surface geometry, circulation and loads, when present |
| `solution/fvm/`, `solution/vpm/`, `solution/vlm/` | The individual frames referenced by the `.pvd` files |
| `solution/*_metadata.json` | The configuration that ran and its timing |
| `solution/*.log` | Solver logs |
| `solution/coupler_diagnostics.jsonl` | Coupled cases: per-exchange boundary convergence, particle counts and timings |
| `samples/` | Sampler histories as CSV: forces, line and plane probes, global invariants |
| `figures/` | Plots written by `./allplot.sh` |
| `assets/<case_name>_mesh.npz` | The generated FVM mesh (with a `.vtu` copy), reused while its inputs are unchanged |

Open a root `.pvd` file in ParaView to load every saved time. Copy the whole `solution/` folder when moving results, so the relative frame paths stay valid. Backups for restarting are described in continuation; field units are in the FVM and VPM guides.

CHAPTER 9

# Inspect and compare the flow

Each tutorial plots its own results:

```
./allplot.sh          # PNG and PDF
./allplot.sh png      # PNG only
./allplot.sh pdf      # PDF only
```

The plot scripts in `assets/` read `samples/` and `solution/` directly; edit them to change a comparison. For fields, open `solution/fvm.pvd`, `solution/vpm.pvd` or `solution/vlm.pvd` in ParaView; several cases provide `assets/paraview_state.py` with a ready-made view. Perceptually uniform colour maps for ParaView are in `docs/assets/scientific_colour_maps/`.

## 9.1 Reading the fields

| Quantity | Interpretation |
|---|---|
| Velocity $\mathbf{u}$, m/s | Flow direction, wake deficit, near-wall profiles |
| Kinematic pressure $q = p/\rho$, m²/s² | Multiply by $\rho$ for pressure in Pa |
| Vorticity $\omega$, 1/s | Local rotation; signed components show its sense |
| Particle strength $\Gamma$, m³/s | Vorticity integrated over a particle volume, not a pointwise value |
| $C_D$, $C_L$ | Force divided by $\frac{1}{2}\rho U^2 A$ with the case's reference area $A$ |

When comparing runs, keep the colour limits, physical times, normalization and averaging windows the same. Use a sequential colour map for magnitudes and a diverging map centred on zero for signed fields. A smooth-looking wake does not prove accuracy: compare forces, profiles or vortex trajectories while refining the resolution and time step.

Your own plots can share the tutorial style through `openonda.plotting` (`set_style()`, `export_figure()`).